

Install Burp Suite Community Edition

install:

- **Burp Suite Community Edition**
- **Firefox (recommended)**

Capture Your First Request

Use a test website or your own practice application.

- 1. Turn Intercept ON**
- 2. Visit a webpage**
- 3. Capture the request**
- 4. Observe:**
 - **URL**
 - **Method**
 - **Headers**
 - **Cookies**

Repeater

- 1. Send request to Repeater.**
- 2. Change harmless values such as:**
 - **Search keyword**
 - **Page number**
 - **Language parameter**
- 3. Resend request.**
- 4. Observe response changes.**

Assignment

Choose a website you are authorized to test.

Capture:

- 1. Homepage Request**
- 2. Login Page Request**
- 3. Search Request**

For each request explain:

- **Method (GET/POST)**
- **URL**
- **Headers**
- **Parameters**
- **Response Code**

Take screenshots and prepare a report.

Aim

To install and configure Burp Suite Community Edition, intercept HTTP/HTTPS traffic, analyze web requests and responses, and understand the communication process between a client browser and a web server.

Objectives

1. To install Burp Suite Community Edition.
2. To configure Mozilla Firefox to work with Burp Suite.
3. To intercept and inspect HTTP requests.
4. To analyze request methods, URLs, headers, and parameters.
5. To understand GET and POST request mechanisms.
6. To use Burp Suite for web traffic monitoring and analysis.

Software and Tools Required

Sr. No.	Software	Purpose
1	Burp Suite Community Edition	Intercepting and analyzing web traffic
2	Mozilla Firefox	Web browser for generating requests
3	Internet Connection	Accessing testing websites
4	HTTPBin.org	Safe testing website

Theory

Burp Suite is a web application security testing platform developed by PortSwigger. It acts as an intercepting proxy between the browser and the target web server. All requests sent from the browser pass through Burp Suite, allowing security analysts to inspect, modify, and forward requests.

HTTP (HyperText Transfer Protocol) is the protocol used for communication between web browsers and web servers. Each request consists of a method, URL, headers, and optional parameters.

The most common HTTP methods are:

GET Method

The GET method is used to retrieve information from a server. Parameters are usually visible in the URL.

Example:

<https://httpbin.org/get?name=tushar&city=jamnagar>

POST Method

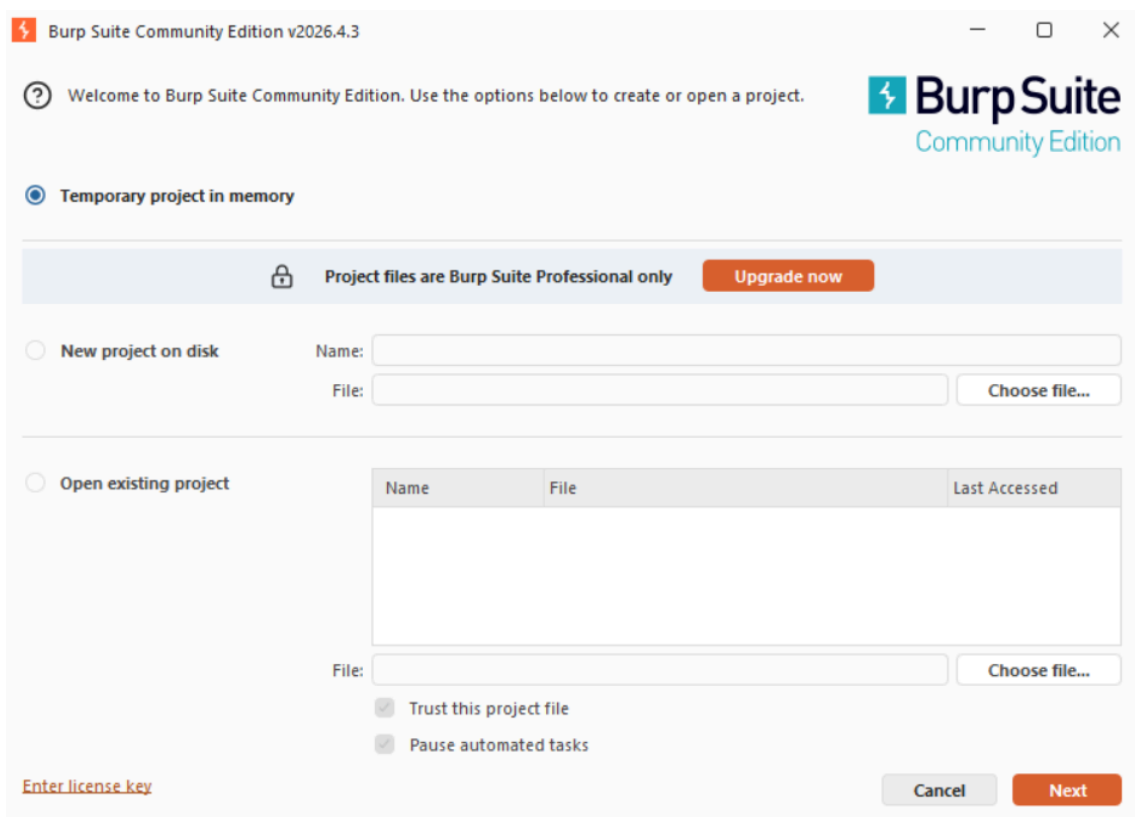
The POST method is used to send data to a server. The data is included in the request body and is commonly used in forms and authentication systems.

Procedure

Step 1: Installation of Burp Suite

1. Download Burp Suite Community Edition.
2. Install the application.
3. Launch Burp Suite.
4. Select "Temporary Project".
5. Choose "Use Burp Defaults".
6. Start Burp Suite.

Figure 1: Burp Suite Temporary Project Screen



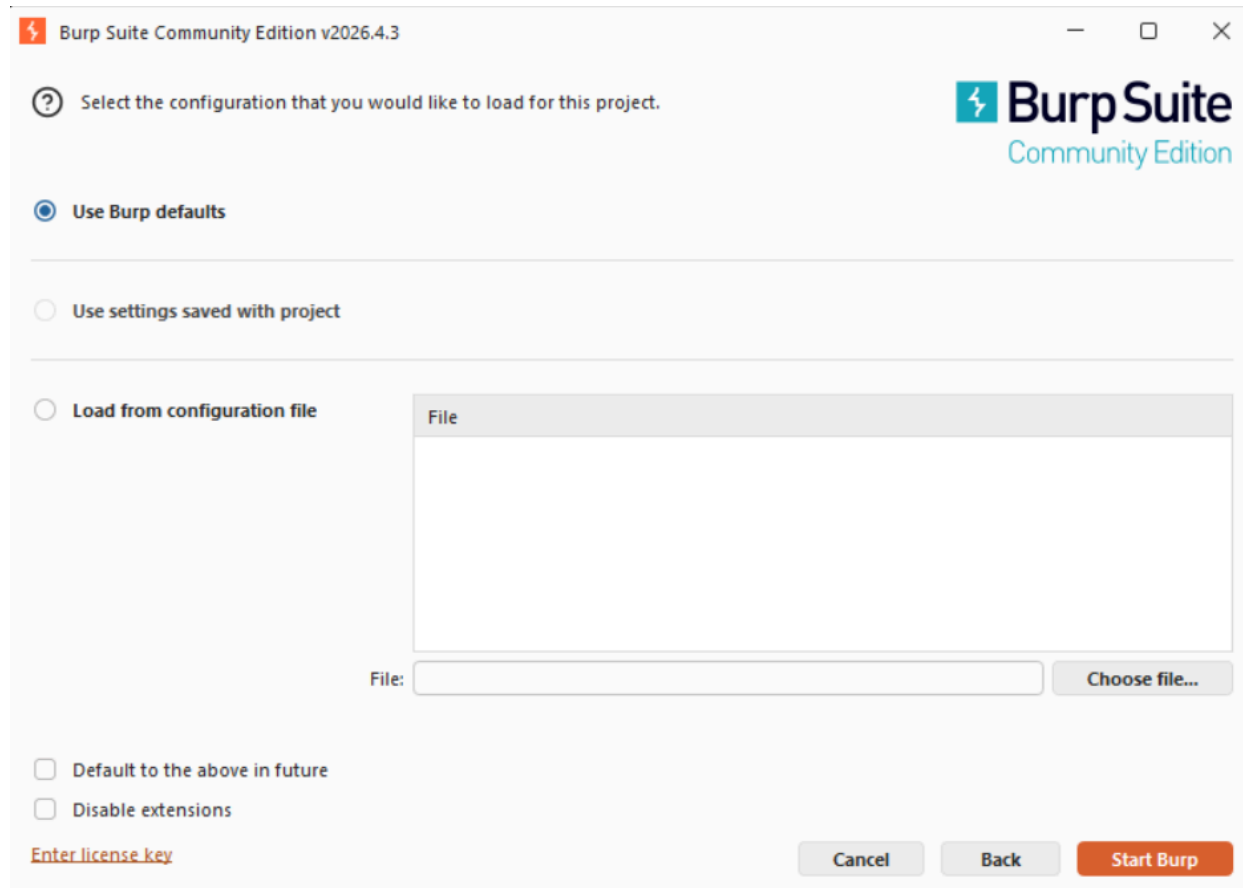


Figure 2: Burp Suite Default Configuration Screen

Step 2: Browser Configuration

1. Open Mozilla Firefox.
2. Navigate to Network Settings.
3. Select Manual Proxy Configuration.

4. Configure:

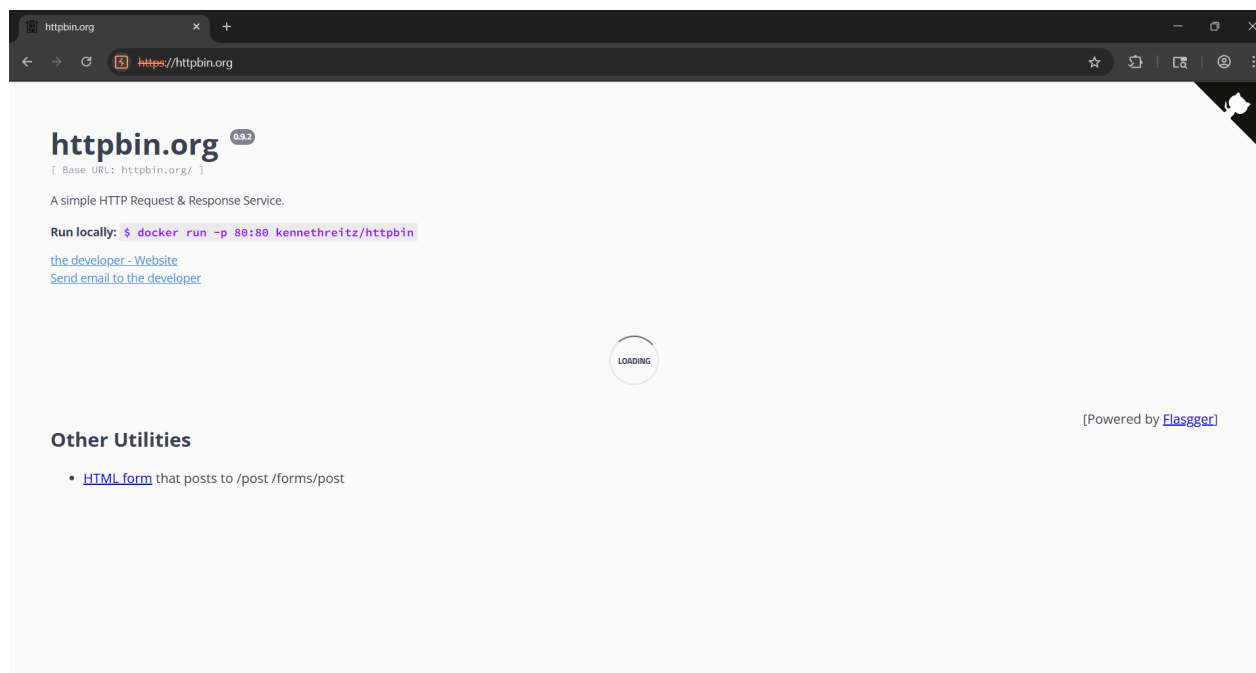
Setting	Value
HTTP Proxy	127.0.0.1
Port	8081

5. Save the configuration.

Step 3: Capturing Homepage Request

1. Enable Intercept mode in Burp Suite.
2. Open <https://httpbin.org>.
3. Burp Suite intercepts the request before forwarding it to the server.
4. Observe request details.

Figure 3: Homepage Request Captured in Burp Suite



Observation

Parameter	Value
Method	GET
URL	https://httpbin.org
Protocol	HTTP/2
Host	httpbin.org
Purpose	Retrieve homepage content

Step 4: Capturing and Modifying a GET Request

The following URL was accessed:

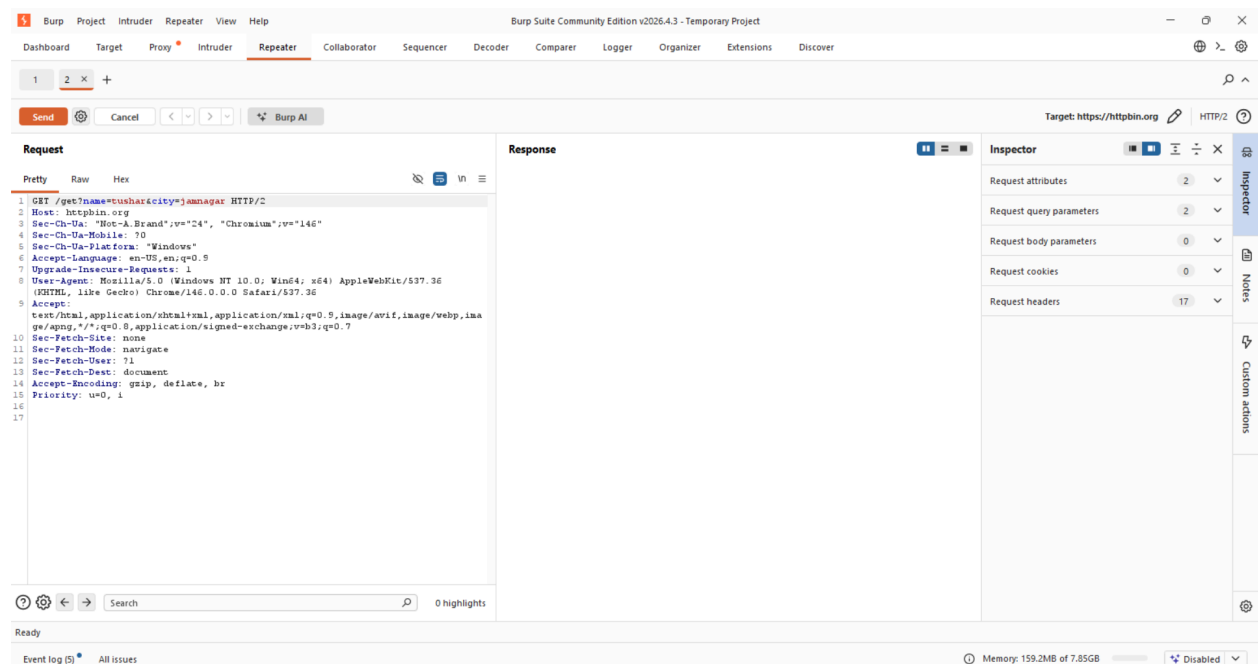
<https://httpbin.org/get?name=tushar&city=jamnagar>

Burp Suite intercepted the request and displayed the parameters before sending them to the server.

Original Request

Parameter Name	Value
name	tushar
city	jamnagar

Figure 4: GET Request with Parameters



Modifying the Request

While the request was intercepted in Burp Suite, the parameter values were manually changed:

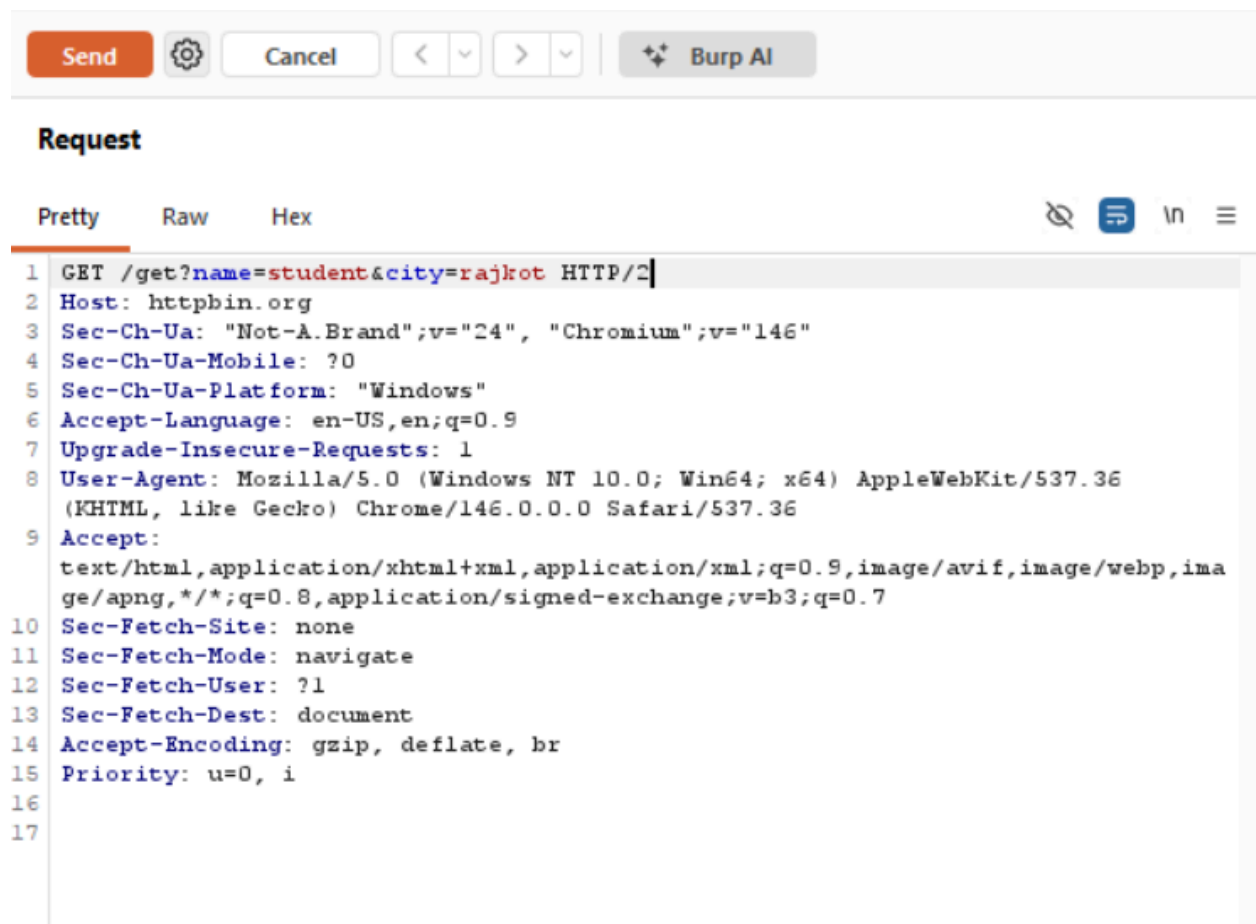
Parameter Name	Original Value	Modified Value
name	tushar	student
city	jamnagar	rajkot

The modified request became:

<https://httpbin.org/get?name=student&city=rajkot>

After editing the values, the Send/Forward button was clicked to send the modified request to the server.

Figure 5: Modified GET Request Before Sending



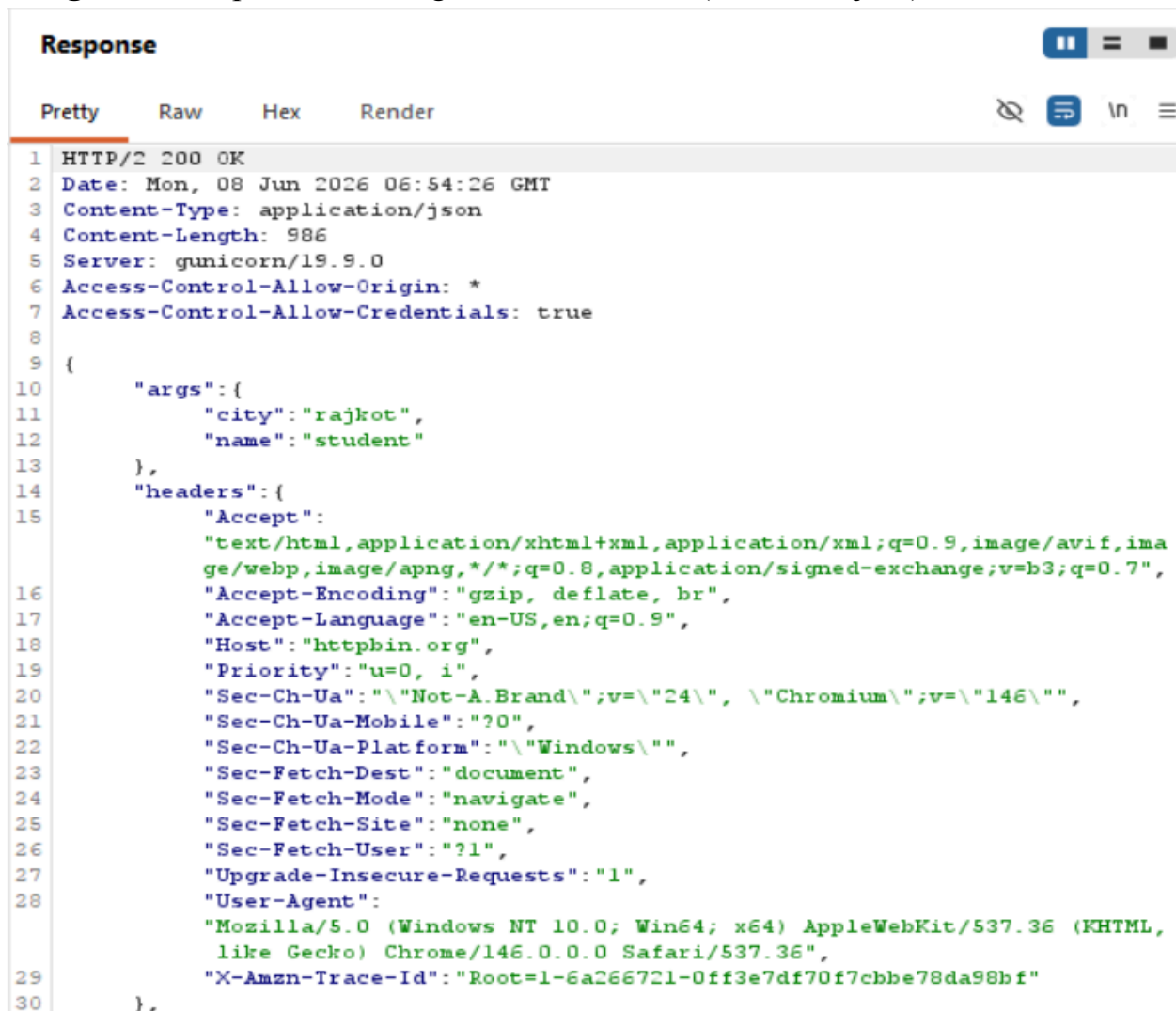
Response Analysis

The server processed the modified request and returned the updated parameter values in the response.

Parameter Name	Response Value
name	student
city	rajkot

This demonstrated that Burp Suite can intercept and alter HTTP requests before they reach the server

Figure 6: Response Showing Modified Values (student, rajkot)

The image shows the 'Response' tab in Burp Suite. The response is an HTTP/2 200 OK status with a JSON body. The JSON body contains an 'args' object with 'city' set to 'rajkot' and 'name' set to 'student'. The 'headers' object lists various headers including 'Accept', 'Accept-Encoding', 'Accept-Language', 'Host', 'Priority', 'Sec-Ch-Ua', 'Sec-Ch-Ua-Mobile', 'Sec-Ch-Ua-Platform', 'Sec-Fetch-Dest', 'Sec-Fetch-Mode', 'Sec-Fetch-Site', 'Sec-Fetch-User', 'Upgrade-Insecure-Requests', and 'User-Agent'.

```
1 HTTP/2 200 OK
2 Date: Mon, 08 Jun 2026 06:54:26 GMT
3 Content-Type: application/json
4 Content-Length: 986
5 Server: gunicorn/19.9.0
6 Access-Control-Allow-Origin: *
7 Access-Control-Allow-Credentials: true
8
9 {
10     "args": {
11         "city": "rajkot",
12         "name": "student"
13     },
14     "headers": {
15         "Accept":
16             "text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7",
17         "Accept-Encoding": "gzip, deflate, br",
18         "Accept-Language": "en-US,en;q=0.9",
19         "Host": "httpbin.org",
20         "Priority": "u=0, i",
21         "Sec-Ch-Ua": "\"Not-A.Brand\";v=\"24\", \"Chromium\";v=\"146\"",
22         "Sec-Ch-Ua-Mobile": "?0",
23         "Sec-Ch-Ua-Platform": "\"Windows\"",
24         "Sec-Fetch-Dest": "document",
25         "Sec-Fetch-Mode": "navigate",
26         "Sec-Fetch-Site": "none",
27         "Sec-Fetch-User": "?1",
28         "Upgrade-Insecure-Requests": "1",
29         "User-Agent":
30             "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36",
31         "X-Amzn-Trace-Id": "Root=1-6a266721-0ff3e7df70f7cbbe78da98bf"
32     }
33 }
```

Request Analysis

Field	Description
Method	GET
URL	/get
Original Parameters	name=tushar, city=jamnagar
Modified Parameters	name=student, city=rajkot
Response	JSON Data
Status Code	200 OK

The server returned a successful response containing the supplied parameter values.

Step 5: POST Request Analysis Using Pizza Order Form

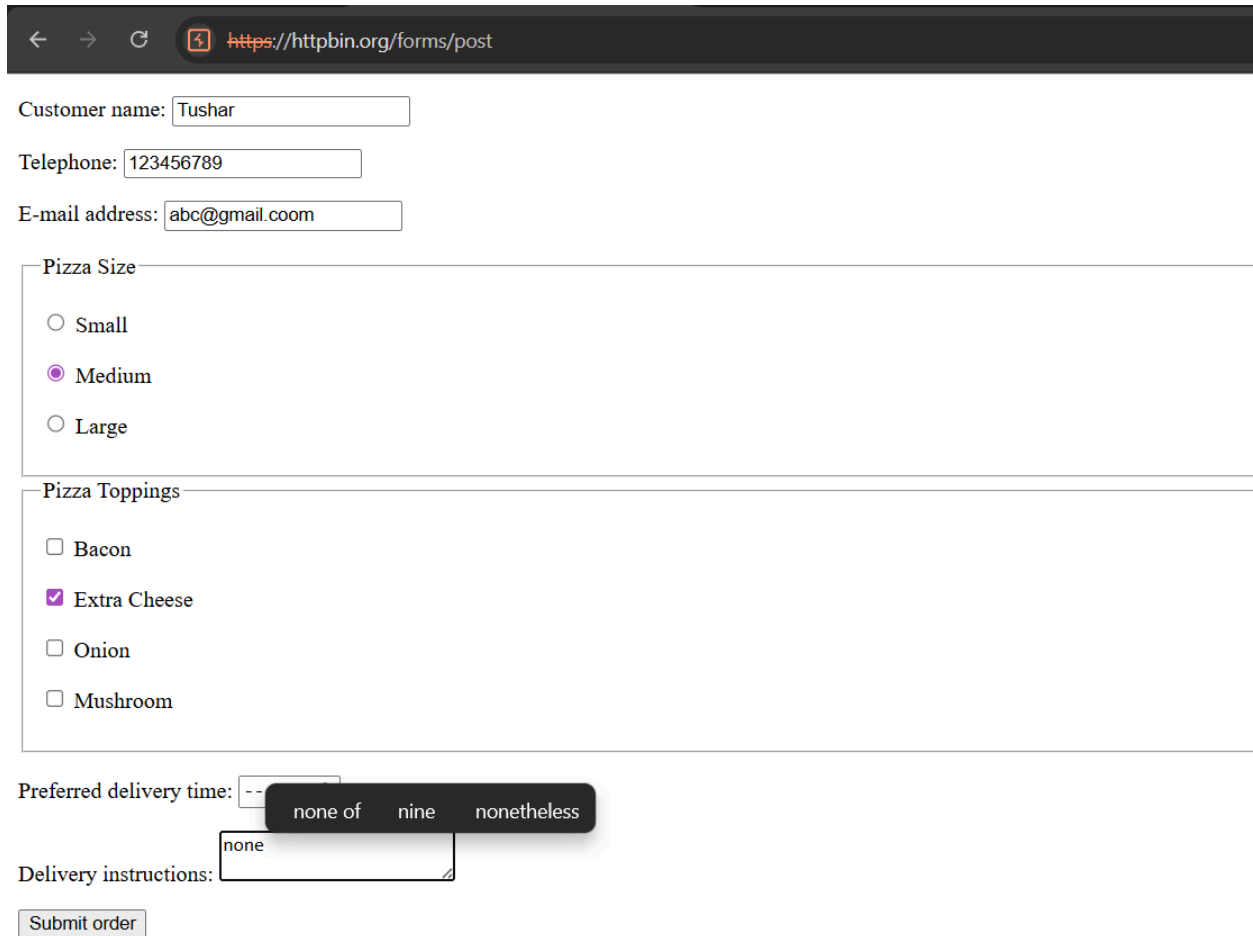
The Pizza Order Form available on HTTPBin was used to generate a POST request.

Procedure:

1. Open the Pizza Order Form page.
2. Enter sample values.
3. Submit the form.
4. Capture the POST request using Burp Suite.

Figure 7: POST Request Analysis Using Pizza Order Form

The Pizza Order Form available on the testing website was submitted to generate a POST request. The request parameters entered in the form were transmitted to the server using the HTTP POST method. Burp Suite successfully intercepted and displayed the submitted data for analysis.



← → ↻ <https://httpbin.org/forms/post>

Customer name:

Telephone:

E-mail address:

Pizza Size

☐ Small

☒ Medium

☐ Large

Pizza Toppings

☐ Bacon

☒ Extra Cheese

☐ Onion

☐ Mushroom

Preferred delivery time:

none of nine nonetheless

Delivery instructions:

Figure 8: Captured POST Request

Request

Pretty Raw Hex

```
1 POST /post HTTP/2
2 Host: httpbin.org
3 Content-Length: 111
4 Cache-Control: max-age=0
5 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
6 Sec-Ch-Ua-Mobile: ?0
7 Sec-Ch-Ua-Platform: "Windows"
8 Accept-Language: en-US,en;q=0.9
9 Origin: https://httpbin.org
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
    (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
13 Accept:
    text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,ima
    ge/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: https://httpbin.org/forms/post
19 Accept-Encoding: gzip, deflate, br
20 Priority: u=0, i
21
22 custname=Tushar&custtel=123456789&custemail=abc%40gmail.com&size=medium&
    topping=cheese&delivery=&comments=none
```

Observation

Field	Value
Method	POST
Content Type	Form Data
Parameters	User Input Values
Purpose	Submit Data to Server

Difference Between GET and POST

Feature	GET	POST
Purpose	Retrieve Data	Submit Data
Data Location	URL	Request Body
Visibility	Visible in URL	Hidden from URL
Data Size	Limited	Large Data Supported

Security	Less Secure	More Secure
----------	-------------	-------------

Request Header Analysis

The captured requests contained several HTTP headers.

Common Headers Observed

Header	Purpose
Host	Specifies target server
User-Agent	Browser information
Accept	Accepted content types
Connection	Connection management
Content-Type	Format of submitted data

Headers provide important information required for successful communication between browser and server.

Results

The following activities were successfully performed:

Activity	Status
Homepage Request Capture	Successful
GET Request Capture	Successful
GET Request Modification	Successful
Verification of Modified Response	Successful
POST Request Capture	Successful

The experiment demonstrated that Burp Suite can intercept, inspect, modify, and forward HTTP requests before they reach the server.

Learning Outcomes

After completing this practical:

1. Learned to configure Burp Suite.
2. Understood browser proxy settings.
3. Captured HTTP and HTTPS traffic.
4. Analyzed GET requests.
5. Modified intercepted request parameters.

6. Verified server responses after modification.
7. Analyzed POST requests.
8. Examined request headers and parameters.
9. Understood client-server communication.
10. Gained hands-on experience with web application traffic analysis.

Conclusion

The practical was successfully completed using Burp Suite Community Edition and Mozilla Firefox. Various HTTP requests were intercepted and analyzed, including homepage requests, parameter-based GET requests, modified GET requests, and POST form submissions. The parameters originally sent as **name=tushar** and **city=jamnagar** were modified to **name=student** and **city=rajkot** before forwarding the request to the server. The server response reflected the modified values, demonstrating how Burp Suite can be used to inspect and manipulate web traffic. This exercise provided practical understanding of request interception, modification, forwarding, and response analysis in web applications.